

Stringhe

Può capitarci di dover gestire delle stringhe di testo. Il linguaggio C, purtroppo, non mette a disposizione il tipo stringa per le variabili.

Questo non si significa che non abbiamo la possibilità di gestire delle stringhe di testo; infatti, una stringa non è altro che una sequenza di caratteri. Furbamente, possiamo rappresentare, quindi, una stringa come un array di caratteri (char).

Esempio:

```
char stringa[] = "Ciao";
```

Osserviamo che, a differenza dei caratteri singoli, che richiedono l'apice singolo, le stringhe vengono racchiuse tra doppi apici.

Un altro modo per assegnare una stringa in fase di dichiarazione è il seguente:

```
char stringa[] = {'C', 'i', 'a', 'o', '\0'};
```

dove si vede ancora di più come una stringa sia un array di caratteri.

Questo secondo metodo è molto più scomodo del precedente per due motivi:

- all'aumentare della lunghezza della stringa richiede più tempo e difficoltà nella scrittura
- è necessario indicare il carattere di terminazione della stringa ('\0').

Leggere una stringa

Come per gli altri tipi di dato, anche per le stringhe possiamo effettuare una lettura da tastiera. Esistono diversi modi:

- usare la funzione `scanf()`, usando `%s` come specificatore di formato. Il principale difetto di questo metodo è quello di fermarsi appena trova uno spazio; questo la rende perfetta per leggere una singola parola, ma inadatta a leggere una stringa contenente più parole separate da degli spazi
- usare la funzione `gets()`, che richiede come parametro la variabile dove memorizzare la stringa; la funzione legge una sequenza di caratteri dallo standard input (stdin)
- usare la funzione `fgets()`, che richiede tre parametri; nell'ordine sono la variabile dove memorizzare la stringa, la quantità di byte da leggere ed il buffer di lettura. Questa funzione in genere è utilizzata per la lettura da file, ma indicando stdin come buffer di input, si ottiene lo stesso risultato della funzione precedente.

I due metodi più comodi da utilizzare sono il primo ed il secondo.

Esempi:

```
char stringa[100];

scanf("%s", stringa); //singola parola
gets(stringa);
fgets(stringa, sizeof(stringa), stdin);
```

Accedere ad un carattere

Dal momento che le stringhe sono array di caratteri, ciascuno di questi occuperà una ben specifica posizione. Quindi possiamo accedere al singolo carattere della stringa esattamente come accederemmo ad un elemento di un array.

Esempio:

```
char stringa[] = "Ciao a tutti";
char carattere1 = stringa[0];
char carattere2 = stringa[4];
```

In questo esempio `carattere1` conterrà il carattere della stringa in posizione 0, ovvero la 'C', mentre `carattere2` conterrà quello in posizione 4, ovvero ' ' (lo spazio).

Chiaramente è possibile anche modificare un certo carattere nello stesso modo:

```
char stringa[] = "Ciao a tutti";
stringa[0] = 'M';
```

In questo modo il carattere in posizione 0 viene assegnato con la lettera 'M' (la stringa diventa "Miao a tutti").

Manipolazione delle stringhe

Esistono diverse funzioni che ci consentono di lavorare con le stringhe e sono tutte accessibili importando la libreria `string.h`.

Lunghezza di una stringa

Come abbiamo detto, le stringhe non sono altro che array di caratteri (`char`); abbiamo già visto come sia possibile capire la dimensione di un array utilizzando la funzione `sizeof()`.

Questa ci restituisce la dimensione (in byte) del nostro array; un carattere occupa 1 byte di memoria; pertanto, `sizeof()` effettivamente potrebbe restituirci il numero di caratteri presenti nella stringa. Tuttavia, `sizeof()` restituisce sempre la dimensione intesa come spazio di memoria allocato per il nostro array, ma la nostra stringa può non occupare tutto lo spazio allocato.

Esempio:

```
char stringa[100] = "Ciao a tutti";
printf("%d\n", sizeof(stringa));
```

Nell'esempio sopra riportato, la stampa della dimensione della variabile stringa ci restituisce 100, anche se la lunghezza effettiva è molto meno.

Quindi, se vogliamo conoscere la lunghezza effettiva di una stringa, possiamo utilizzare la funzione `strlen()`, che prende come parametro la stringa stessa e restituisce il numero di caratteri contenuti nella stringa.

Sintassi:

```
strlen(stringa);
```

N.B.: `sizeof()` tiene conto anche del carattere di terminazione `'\0'`, mentre `strlen()` non lo considera.

Esempio:

```
char stringa[] = "Ciao a tutti";  
printf("%d\n", sizeof(stringa));  
printf("%d\n", strlen(stringa));
```

In questo caso la prima stampa ci restituisce 13 (12 caratteri + terminazione), mentre la seconda 12 (solo il numero di caratteri).

Confronto tra stringhe

Può capitarci di voler capire se due stringhe siano uguali tra loro oppure no; per farlo abbiamo due possibilità.

La prima è quella di effettuare un confronto manuale tra le due stringhe; per fare questo una prima osservazione che possiamo fare riguarda la lunghezza delle due stringhe: se le due stringhe hanno lunghezze diverse, allora sicuramente non sono uguali.

Nel caso in cui le due stringhe fossero della stessa lunghezza, dobbiamo necessariamente confrontare carattere per carattere le due stringhe: se troviamo una coppia di caratteri diversi tra di loro, allora le due stringhe sono diverse, altrimenti se arriviamo alla fine di entrambe le stringhe senza aver trovato una coppia di caratteri diversi tra di loro possiamo concludere che le due stringhe sono effettivamente uguali.

Non è un procedimento complicato, ma è molto meccanico.

Fortunatamente ci viene in aiuto una funzione, `strcmp()`.

Sintassi:

```
strcmp(stringa1, stringa2);
```

Questa funzione prende come parametri le due stringhe da confrontare e restituisce un valore numerico pari a 0 se queste sono uguali, diverso da 0 se invece sono diverse.

Copiare una stringa

È possibile copiare una stringa all'interno di un'altra. Per farlo adoperiamo la funzione `strcpy()`.

Sintassi:

```
strcpy(stringa2, stringa1);
```

Questa funzione ha bisogno di due parametri: il primo è la stringa di destinazione, il secondo è la stringa di partenza. Quindi, `stringa1` viene copiato all'interno di `stringa2`.

N.B.: è necessario, ovviamente, che la dimensione della seconda stringa sia sufficiente per contenere il contenuto della prima.

Concatenare due stringhe

Spesso può capitarci di voler concatenare due stringhe, ovvero unirle tra di loro in modo da formare un'unica stringa. Questo è possibile grazie alla funzione `strcat()`.

Sintassi:

```
strcat(stringa1, stringa2);
```

La funzione prende come parametri le due stringhe da concatenare e memorizza il risultato nella prima stringa (`stringa1`).

Maiuscolo e minuscolo

Potrebbe essere utile, in alcuni contesti, dover avere una stringa scritta tutta in minuscolo oppure tutta in maiuscolo. Per fare questo esistono due funzioni, contenute nella libreria `ctype.h`:

- `tolower()`, applicata ad un singolo carattere, lo porta in lowercase (minuscolo)

`toupper()`, applicata ad un singolo carattere, lo porta in uppercase (maiuscolo)